BEDROCK

Security Configuration Assessment Report

CIS Kubernetes Benchmark Self-Assessment

RKE2 tenant cluster, benchmark profile rke2-cis-1.9

Document ID	BDR-SA-2026-001
Version	1.0 (Final)
Assessment date	13 October 2026
System	Bedrock tenant cluster, node bedrock-1
Prepared by	sassendes
Classification	Internal
Repository	github.com/sassendes/bedrock

This document records a point-in-time self-assessment performed with open-source tooling. It is not a certification and has not been reviewed, authorized or endorsed by the Center for Internet Security, Inc. See Section 0.3.

0. Document control

0.1 Revision history

Version	Date	Author	Description
1.0	13 October 2026	sassendes	Initial release.

0.2 Approval

Role	Name	Signature	Date
Prepared by	sassendes	*/s/ sassendes*	13 October 2026
Independent review	Not performed (self-assessment)	n/a	n/a

0.3 Notices

Trademarks. CIS Benchmarks™ is a trademark of the Center for Internet Security, Inc. (CIS). This report is an independent publication and has not been authorized, sponsored or otherwise approved by CIS. Nothing in it states or implies CIS certification or endorsement of Bedrock. Other product names are the property of their respective owners.

Tooling. Results were produced by kube-bench, an open-source tool published by Aqua Security under the Apache License 2.0, using the rke2-cis-1.9 benchmark profile. kube-bench is not itself certified by CIS.

Limitations. The assessment reflects the configuration of the system under test on the assessment date only. It covers the controls automated or listed by the tool and does not constitute a penetration test, a code review, or an audit of applications running on the cluster. Results may differ on other nodes or after configuration changes.

Use. This document is provided for information. It is supplied as-is, without warranty of any kind.

1. Executive summary

A Bedrock tenant cluster running RKE2 with the CIS profile was assessed against the CIS Kubernetes Benchmark v1.9 as profiled for RKE2. Of 129 controls evaluated, 85 passed, 1 failed, 36 require manual review, and 7 are informational.

No failures were recorded against the control plane, etcd or worker node configuration. The single failure (5.1.6) is a workload policy control that flags pods mounting service account tokens; the affected pods are operators and controllers that require API access. It is recorded as accepted by design in Section 4.

Benchmark section	Pass	Fail	Warn	Info
1 Control plane security configuration	56	0	3	0
2 etcd node configuration	7	0	0	0
3 Control plane configuration	1	0	4	0
4 Worker node security configuration	18	0	0	5
5 Kubernetes policies	3	**1**	29	2
Total	**85**	**1**	**36**	**7**

Table 1. Results by benchmark section.

2. Scope and methodology

2.1 System under test

Node	bedrock-1, control plane server (one of three)
Distribution	RKE2 v1.36.4+rke2r1, profile: cis
Container network	Cilium
Benchmark	CIS Kubernetes Benchmark v1.9, RKE2 profile (rke2-cis-1.9)
Tool	kube-bench v0.16.0, run on the node as root
Sections	1 to 5 (control plane, etcd, control plane configuration, worker node, policies)

2.2 Result definitions

Result	Meaning
PASS	Automated control; the system met the expected condition.
FAIL	Automated control; the system did not meet the expected condition.
WARN	Manual control; the tool cannot verify it automatically and operator review is required. A WARN is not a failure.
INFO	Informational; the control could not be evaluated in this environment or is reported for reference only.

3. Findings register

ID	Control	Title	Result	Disposition
F-001	5.1.6	Ensure that service account tokens are only mounted where necessary	**FAIL**	Accepted by design

4. Disposition of F-001

Observation. Control 5.1.6 fails any pod that mounts a service account token without both the pod and its service account disabling automount.

Mitigation in place. The default service account in every namespace has `automountServiceAccountToken: false`. Control 5.1.5 (default service accounts not actively used) passes accordingly.

Rationale. The pods that still mount a token are operators and controllers whose function is to read and write Kubernetes API objects. Each was reviewed and confirmed to require API access; disabling its token would stop it working.

Decision. Accepted as a documented exception. The benchmark configuration was not modified to suppress the result.

5. Manual controls

The 36 WARN results are manual controls. Their status in Bedrock is summarised below; the complete list is in Appendix A.

Controls	Status	Notes
5.2.1 to 5.2.13	**Addressed**	Pod Security Admission enforced per namespace; Kyverno admission policy, fail-closed, on application namespaces.
3.2.2	**Addressed**	API audit policy at Metadata level, shipped to Loki.
5.3.2	**Planned**	Cilium default-deny NetworkPolicies, rolled out per namespace.
5.4.2	**Planned**	External secret storage through a per-tenant OpenBao.
Others	**Reviewed**	No action recorded in this version.

Appendix A. Detailed results

Results exactly as reported by the tool, in benchmark order.

1 Control Plane Security Configuration

1.1 Control Plane Node Configuration Files

Control	Title	Type	Result
1.1.1	Ensure that the API server pod specification file permissions are set to 600 or more restrictive	Automated	PASS
1.1.2	Ensure that the API server pod specification file ownership is set to root:root	Automated	PASS
1.1.3	Ensure that the controller manager pod specification file permissions are set to 600 or more restrictive	Automated	PASS
1.1.4	Ensure that the controller manager pod specification file ownership is set to root:root	Automated	PASS
1.1.5	Ensure that the scheduler pod specification file permissions are set to 600 or more restrictive	Automated	PASS
1.1.6	Ensure that the scheduler pod specification file ownership is set to root:root	Automated	PASS
1.1.7	Ensure that the etcd pod specification file permissions are set to 600 or more restrictive	Manual	PASS
1.1.8	Ensure that the etcd pod specification file ownership is set to root:root	Manual	PASS
1.1.9	Ensure that the Container Network Interface file permissions are set to 600 or more restrictive	Manual	PASS
1.1.10	Ensure that the Container Network Interface file ownership is set to root:root	Manual	PASS
1.1.11	Ensure that the etcd data directory permissions are set to 700 or more restrictive	Manual	PASS
1.1.12	Ensure that the etcd data directory ownership is set to etcd:etcd	Manual	PASS
1.1.13	Ensure that the admin.conf file permissions are set to 600 or more restrictive	Automated	PASS
1.1.14	Ensure that the admin.conf file ownership is set to root:root	Automated	PASS
1.1.15	Ensure that the scheduler.conf file permissions are set to 600 or more restrictive	Automated	PASS
1.1.16	Ensure that the scheduler.conf file ownership is set to root:root	Automated	PASS
1.1.17	Ensure that the controller-manager.conf file permissions are set to 600 or more restrictive	Automated	PASS
1.1.18	Ensure that the controller-manager.conf file ownership is set to root:root	Automated	PASS
1.1.19	Ensure that the Kubernetes PKI directory and file ownership is set to root:root	Automated	PASS
1.1.20	Ensure that the Kubernetes PKI certificate file permissions are set to 600 or more restrictive	Manual	PASS
1.1.21	Ensure that the Kubernetes PKI key file permissions are set to 600	Automated	PASS

1.2 API Server

Control	Title	Type	Result
1.2.1	Ensure that the --anonymous-auth argument is set to false	Automated	PASS
1.2.2	Ensure that the --token-auth-file parameter is not set	Automated	PASS
1.2.3	Ensure that the --DenyServiceExternalIPs is set	Manual	WARN
1.2.4	Ensure that the --kubelet-client-certificate and --kubelet-client-key arguments are set as appropriate	Automated	PASS

Control	Title	Type	Result
1.2.5	Ensure that the --kubelet-certificate-authority argument is set as appropriate	Automated	PASS
1.2.6	Ensure that the --authorization-mode argument is not set to AlwaysAllow	Automated	PASS
1.2.7	Ensure that the --authorization-mode argument includes Node	Automated	PASS
1.2.8	Ensure that the --authorization-mode argument includes RBAC	Automated	PASS
1.2.9	Ensure that the admission control plugin EventRateLimit is set	Manual	WARN
1.2.10	Ensure that the admission control plugin AlwaysAdmit is not set	Automated	PASS
1.2.11	Ensure that the admission control plugin AlwaysPullImages is set	Manual	WARN
1.2.12	Ensure that the admission control plugin ServiceAccount is set	Automated	PASS
1.2.13	Ensure that the admission control plugin NamespaceLifecycle is set	Automated	PASS
1.2.14	Ensure that the admission control plugin NodeRestriction is set	Automated	PASS
1.2.15	Ensure that the --profiling argument is set to false	Automated	PASS
1.2.16	Ensure that the --audit-log-path argument is set	Automated	PASS
1.2.17	Ensure that the --audit-log-maxage argument is set to 30 or as appropriate	Automated	PASS
1.2.18	Ensure that the --audit-log-maxbackup argument is set to 10 or as appropriate	Automated	PASS
1.2.19	Ensure that the --audit-log-maxsize argument is set to 100 or as appropriate	Automated	PASS
1.2.20	Ensure that the --request-timeout argument is set as appropriate	Automated	PASS
1.2.21	Ensure that the --service-account-lookup argument is set to true	Automated	PASS
1.2.22	Ensure that the --service-account-key-file argument is set as appropriate	Automated	PASS
1.2.23	Ensure that the --etcd-certfile and --etcd-keyfile arguments are set as appropriate	Automated	PASS
1.2.24	Ensure that the --tls-cert-file and --tls-private-key-file arguments are set as appropriate	Automated	PASS
1.2.25	Ensure that the --client-ca-file argument is set as appropriate	Automated	PASS
1.2.26	Ensure that the --etcd-cafile argument is set as appropriate	Automated	PASS
1.2.27	Ensure that the --encryption-provider-config argument is set as appropriate	Automated	PASS
1.2.28	Ensure that encryption providers are appropriately configured	Automated	PASS
1.2.29	Ensure that the API Server only makes use of Strong Cryptographic Ciphers	Automated	PASS

1.3 Controller Manager

Control	Title	Type	Result
1.3.1	Ensure that the --terminated-pod-gc-threshold argument is set as appropriate	Manual	PASS
1.3.2	Ensure that the --profiling argument is set to false	Automated	PASS
1.3.3	Ensure that the --use-service-account-credentials argument is set to true	Automated	PASS
1.3.4	Ensure that the --service-account-private-key-file argument is set as appropriate	Automated	PASS
1.3.5	Ensure that the --root-ca-file argument is set as appropriate	Automated	PASS
1.3.6	Ensure that the RotateKubeletServerCertificate argument is set to true	Automated	PASS
1.3.7	Ensure that the --bind-address argument is set to 127.0.0.1	Automated	PASS

1.4 Scheduler

Control	Title	Type	Result
1.4.1	Ensure that the --profiling argument is set to false	Automated	PASS
1.4.2	Ensure that the --bind-address argument is set to 127.0.0.1	Automated	PASS

2 Etcd Node Configuration

Control	Title	Type	Result
2.1	Ensure that the --cert-file and --key-file arguments are set as appropriate	Automated	PASS
2.2	Ensure that the --client-cert-auth argument is set to true	Automated	PASS
2.3	Ensure that the --auto-tls argument is not set to true	Automated	PASS
2.4	Ensure that the --peer-cert-file and --peer-key-file arguments are set as appropriate	Automated	PASS
2.5	Ensure that the --peer-client-cert-auth argument is set to true	Automated	PASS
2.6	Ensure that the --peer-auto-tls argument is not set to true	Automated	PASS
2.7	Ensure that a unique Certificate Authority is used for etcd	Automated	PASS

3 Control Plane Configuration

3.1 Authentication and Authorization

Control	Title	Type	Result
3.1.1	Client certificate authentication should not be used for users	Manual	WARN
3.1.2	Service account token authentication should not be used for users	Manual	WARN
3.1.3	Bootstrap token authentication should not be used for users	Manual	WARN

3.2 Logging

Control	Title	Type	Result
3.2.1	Ensure that a minimal audit policy is created	Automated	PASS
3.2.2	Ensure that the audit policy covers key security concerns	Manual	WARN

4 Worker Node Security Configuration

4.1 Worker Node Configuration Files

Control	Title	Type	Result
4.1.1	Ensure that the kubelet service file permissions are set to 600 or more restrictive	Automated	INFO
4.1.2	Ensure that the kubelet service file ownership is set to root:root	Automated	INFO
4.1.3	If proxy kubeconfig file exists ensure permissions are set to 600 or more restrictive	Automated	PASS
4.1.4	If proxy kubeconfig file exists ensure ownership is set to root:root	Automated	PASS
4.1.5	Ensure that the --kubeconfig kubelet.conf file permissions are set to 600 or more restrictive	Automated	PASS
4.1.6	Ensure that the --kubeconfig kubelet.conf file ownership is set to root:root	Automated	PASS
4.1.7	Ensure that the certificate authorities file permissions are set to 600 or more restrictive	Automated	PASS
4.1.8	Ensure that the client certificate authorities file ownership is set to root:root	Automated	PASS

Control	Title	Type	Result
4.1.9	If the kubelet config.yaml configuration file is being used validate permissions set to 600 or more restrictive	Automated	INFO
4.1.10	If the kubelet config.yaml configuration file is being used validate file ownership is set to root:root	Automated	INFO

4.2 Kubelet

Control	Title	Type	Result
4.2.1	Ensure that the --anonymous-auth argument is set to false	Automated	PASS
4.2.2	Ensure that the --authorization-mode argument is not set to AlwaysAllow	Automated	PASS
4.2.3	Ensure that the --client-ca-file argument is set as appropriate	Automated	PASS
4.2.4	Verify that the --read-only-port argument is set to 0	Automated	PASS
4.2.5	Ensure that the --streaming-connection-idle-timeout argument is not set to 0	Manual	PASS
4.2.6	Ensure that the --make-iptables-util-chains argument is set to true	Automated	PASS
4.2.7	Ensure that the --hostname-override argument is not set	Automated	INFO
4.2.8	Ensure that the eventRecordQPS argument is set to a level which ensures appropriate event capture	Manual	PASS
4.2.9	Ensure that the --tls-cert-file and --tls-private-key-file arguments are set as appropriate	Automated	PASS
4.2.10	Ensure that the --rotate-certificates argument is not set to false	Automated	PASS
4.2.11	Verify that the RotateKubeletServerCertificate argument is set to true	Automated	PASS
4.2.12	Ensure that the Kubelet only makes use of Strong Cryptographic Ciphers	Manual	PASS
4.2.13	Ensure that a limit is set on pod PIDs	Manual	PASS

5 Kubernetes Policies

5.1 RBAC and Service Accounts

Control	Title	Type	Result
5.1.1	Ensure that the cluster-admin role is only used where required	Automated	PASS
5.1.2	Minimize access to secrets	Automated	INFO
5.1.3	Minimize wildcard use in Roles and ClusterRoles	Automated	PASS
5.1.4	Minimize access to create pods	Automated	INFO
5.1.5	Ensure that default service accounts are not actively used.	Automated	PASS
5.1.6	Ensure that Service Account Tokens are only mounted where necessary	Automated	**FAIL**
5.1.7	Avoid use of system:masters group	Manual	WARN
5.1.8	Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster	Manual	WARN
5.1.9	Minimize access to create persistent volumes	Manual	WARN
5.1.10	Minimize access to the proxy sub-resource of nodes	Manual	WARN
5.1.11	Minimize access to the approval sub-resource of certificatesigningrequests objects	Manual	WARN

Control	Title	Type	Result
5.1.12	Minimize access to webhook configuration objects	Manual	WARN
5.1.13	Minimize access to the service account token creation	Manual	WARN

5.2 Pod Security Standards

Control	Title	Type	Result
5.2.1	Ensure that the cluster has at least one active policy control mechanism in place	Manual	WARN
5.2.2	Minimize the admission of privileged containers	Manual	WARN
5.2.3	Minimize the admission of containers wishing to share the host process ID namespace	Manual	WARN
5.2.4	Minimize the admission of containers wishing to share the host IPC namespace	Manual	WARN
5.2.5	Minimize the admission of containers wishing to share the host network namespace	Manual	WARN
5.2.6	Minimize the admission of containers with allowPrivilegeEscalation	Manual	WARN
5.2.7	Minimize the admission of root containers	Manual	WARN
5.2.8	Minimize the admission of containers with the NET_RAW capability	Manual	WARN
5.2.9	Minimize the admission of containers with added capabilities	Manual	WARN
5.2.10	Minimize the admission of containers with capabilities assigned	Manual	WARN
5.2.11	Minimize the admission of Windows HostProcess containers	Manual	WARN
5.2.12	Minimize the admission of HostPath volumes	Manual	WARN
5.2.13	Minimize the admission of containers which use HostPorts	Manual	WARN

5.3 Network Policies and CNI

Control	Title	Type	Result
5.3.1	Ensure that the CNI in use supports NetworkPolicies	Manual	WARN
5.3.2	Ensure that all Namespaces have NetworkPolicies defined	Manual	WARN

5.4 Secrets Management

Control	Title	Type	Result
5.4.1	Prefer using Secrets as files over Secrets as environment variables	Manual	WARN
5.4.2	Consider external secret storage	Manual	WARN

5.5 Extensible Admission Control

Control	Title	Type	Result
5.5.1	Configure Image Provenance using ImagePolicyWebhook admission controller	Manual	WARN

5.7 General Policies

Control	Title	Type	Result
5.7.1	Create administrative boundaries between resources using namespaces	Manual	WARN
5.7.2	Ensure that the seccomp profile is set to docker/default in your Pod definitions	Manual	WARN
5.7.3	Apply SecurityContext to your Pods and Containers	Manual	WARN
5.7.4	The default namespace should not be used	Manual	WARN

Appendix B. Tool remediation guidance

Reproduced verbatim from the tool output for each non-passing control.

1 Control plane security configuration

```
1.2.3 By default, RKE2 does not set DenyServiceExternalIPs.
To enable this flag, edit the RKE2 config file /etc/rancher/rke2/config.yaml like below.
kube-apiserver-arg:
  - "enable-admission-plugins=DenyServiceExternalIPs"
```

```
1.2.9 Follow the Kubernetes documentation and set the desired limits in a configuration file.
Then, edit the RKE2 config file /etc/rancher/rke2/config.yaml and set the below parameters.
kube-apiserver-arg:
  - "enable-admission-plugins=...,EventRateLimit,..."
  - "admission-control-config-file=<path/to/configuration/file>"
```

```
1.2.11 Permissive, per CIS guidelines,
"This setting could impact offline or isolated clusters, which have images pre-loaded and
do not have access to a registry to pull in-use images. This setting is not appropriate for
clusters which use this configuration."
Edit the RKE2 config file /etc/rancher/rke2/config.yaml
on the control plane node and set the --enable-admission-plugins parameter to include
AlwaysPullImages.
--enable-admission-plugins=...,AlwaysPullImages,...
```

3 Control plane configuration

```
3.1.1 Alternative mechanisms provided by Kubernetes such as the use of OIDC should be
implemented in place of client certificates.
```

```
3.1.2 Alternative mechanisms provided by Kubernetes such as the use of OIDC should be implemented
in place of service account tokens.
```

```
3.1.3 Alternative mechanisms provided by Kubernetes such as the use of OIDC should be implemented
in place of bootstrap tokens.
```

```
3.2.2 Review the audit policy provided for the cluster and ensure that it covers
at least the following areas,
- Access to Secrets managed by the cluster. Care should be taken to only
  log Metadata for requests to Secrets, ConfigMaps, and TokenReviews, in
  order to avoid risk of logging sensitive data.
- Modification of Pod and Deployment objects.
- Use of `pods/exec`, `pods/portforward`, `pods/proxy` and `services/proxy`.
For most requests, minimally logging at the Metadata level is recommended
(the most basic level of logging).
```

5 Kubernetes policies

```
5.1.6 Modify the definition of ServiceAccounts and Pods which do not need to mount service
account tokens to disable it, with `automountServiceAccountToken: false`.
RKE2 gives exceptions to the ServiceAccounts in kube-system namespace, as these are expected contact the
apiserver.
When in cis mode, RKE2 disables the automountServiceAccountToken for namespaces:
- kube-public, default, kube-node-lease
If both the ServiceAccount and the Pod's .spec specify a value for automountServiceAccountToken, the Pod
spec takes precedence.
Condition: Pod is_compliant to true when
  - ServiceAccount is automountServiceAccountToken: false and Pod is automountServiceAccountToken: false
or notset
  - ServiceAccount is automountServiceAccountToken: true/notset and Pod is automountServiceAccountToken:
false
```

```
5.1.7 Remove the system:masters group from all users in the cluster.
```

```
5.1.8 Where possible, remove the impersonate, bind and escalate rights from subjects.
```

```
5.1.9 Where possible, remove create access to PersistentVolume objects in the cluster.
```

```
5.1.10 Where possible, remove access to the proxy sub-resource of node objects.
```

```
5.1.11 Where possible, remove access to the approval sub-resource of certificatesigningrequest objects.
```

```
5.1.12 Where possible, remove access to the validatingwebhookconfigurations or
mutatingwebhookconfigurations objects
```

```
5.1.13 Where possible, remove access to the token sub-resource of serviceaccount objects.
```

5.2.1 Ensure that either Pod Security Admission or an external policy control system is in place for every namespace which contains user workloads.

5.2.2 Add policies to each namespace in the cluster which has user workloads to restrict the admission of privileged containers.

5.2.3 Add policies to each namespace in the cluster which has user workloads to restrict the admission of `hostPID` containers.

5.2.4 Add policies to each namespace in the cluster which has user workloads to restrict the admission of `hostIPC` containers.

5.2.5 Add policies to each namespace in the cluster which has user workloads to restrict the admission of `hostNetwork` containers.

5.2.6 Add policies to each namespace in the cluster which has user workloads to restrict the admission of containers with `.spec.allowPrivilegeEscalation` set to `true`.

5.2.7 Create a policy for each namespace in the cluster, ensuring that either `MustRunAsNonRoot` or `MustRunAs` with the range of UIDs not including 0, is set.

5.2.8 Add policies to each namespace in the cluster which has user workloads to restrict the admission of containers with the `NET_RAW` capability.

5.2.9 Ensure that `allowedCapabilities` is not present in policies for the cluster unless it is set to an empty array.

5.2.10 Review the use of capabilities in applications running on your cluster. Where a namespace contains applications which do not require any Linux capabities to operate consider adding a PSP which forbids the admission of containers which do not drop all capabilities.

5.2.11 Add policies to each namespace in the cluster which has user workloads to restrict the admission of containers that have `.securityContext.windowsOptions.hostProcess` set to `true`.

5.2.12 Add policies to each namespace in the cluster which has user workloads to restrict the admission of containers with `hostPath` volumes.

5.2.13 Add policies to each namespace in the cluster which has user workloads to restrict the admission of containers which use `hostPort` sections.

5.3.1 If the CNI plugin in use does not support network policies, consideration should be given to making use of a different plugin, or finding an alternate mechanism for restricting traffic in the Kubernetes cluster.

5.3.2 Follow the documentation and create NetworkPolicy objects as you need them.

5.4.1 If possible, rewrite application code to read Secrets from mounted secret files, rather than from environment variables.

5.4.2 Refer to the Secrets management options offered by your cloud provider or a third-party secrets management solution.

5.5.1 Follow the Kubernetes documentation and setup image provenance.

5.7.1 Follow the documentation and create namespaces for objects in your deployment as you need them.

5.7.2 Use `securityContext` to enable the docker/default seccomp profile in your pod definitions. An example is as below:

```
  securityContext:
    seccompProfile:
      type: RuntimeDefault
```

5.7.3 Follow the Kubernetes documentation and apply SecurityContexts to your Pods. For a suggested list of SecurityContexts, you may refer to the CIS Security Benchmark for Docker Containers.

5.7.4 Ensure that namespaces are created to allow for appropriate segregation of Kubernetes resources and that all new resources are created in a specific namespace.

Appendix C. References

Ref	Document
[1]	Center for Internet Security. CIS Kubernetes Benchmark v1.9. cisecurity.org
[2]	RKE2 documentation. CIS 1.9 Self-Assessment Guide and CIS Hardening Guide. docs.rke2.io/security
[3]	Aqua Security. kube-bench. github.com/aquasecurity/kube-bench
[4]	Bedrock platform specification and source. github.com/sassendes/bedrock

End of document